

INSPER

Engenharia de Computação

Computação em Nuvem — 2026/1

CloudPay

Plataforma Escalável de Pagamentos em Nuvem

Relatório Técnico — Projeto Final

Lucas Kenji Kamikawa | Gabriel Vidigal

lucaskmk@al.insper.edu.br | gabrielcv1@al.insper.edu.br

Maio de 2026

Sumário

- 1** Resumo Executivo
- 2** Contexto e Motivação
- 3** Arquitetura da Solução
 - 3.1** Visão Geral
 - 3.2** Fluxo Completo de uma Transação
 - 3.3** Camadas do Sistema
- 4** Serviços AWS — Implementação e Justificativas
 - 4.1** Amazon API Gateway (HTTP API v2)
 - 4.2** AWS Lambda — 6 Funções Serverless
 - 4.3** Amazon SQS — Processamento Assíncrono
 - 4.4** Amazon DynamoDB — Persistência NoSQL
 - 4.5** Amazon S3 — Hosting do Frontend
 - 4.6** AWS IAM — Segurança e Permissões
 - 4.7** Amazon CloudWatch — Observabilidade
- 5** Frontend — Painel Administrativo
- 6** Testes de Carga com Apache JMeter
- 7** Decisões Técnicas e Trade-offs
- 8** Limitações e Evolução Futura
- 9** Conclusão

1. Resumo Executivo

O CloudPay é um MVP de plataforma de meios de pagamento construído integralmente sobre serviços gerenciados da Amazon Web Services (AWS), na região us-east-2 (Ohio). A solução implementa uma arquitetura serverless orientada a eventos, em que cada transação financeira é processada de forma assíncrona através de uma fila SQS, desacoplando completamente a camada de recepção da camada de processamento.

O sistema é composto por seis funções AWS Lambda escritas em Node.js 22.x, um banco de dados NoSQL DynamoDB com duas tabelas independentes, um Amazon API Gateway (HTTP API v2) que expõe todos os endpoints REST, e uma fila Amazon SQS Standard que medeia o fluxo assíncrono de pagamentos. O painel administrativo foi desenvolvido em React 19 com TypeScript e hospedado como site estático no Amazon S3.

A solução foi validada com testes de carga via Apache JMeter, simulando cenários de alta concorrência que confirmaram a capacidade de absorção de picos sem degradação perceptível na camada de recepção. O sistema encontra-se em operação no endereço: <http://cloudpay-frontend.s3-website.us-east-2.amazonaws.com>

2. Contexto e Motivação

A Demay's Infra Company foi contratada por um cliente do setor financeiro com o objetivo de projetar e entregar uma plataforma digital de meios de pagamento moderna, escalável e baseada em computação em nuvem. O escopo definido para o MVP contempla:

- Cadastro e autenticação de usuários
- Criação e submissão de pagamentos via API
- Processamento assíncrono de transações via fila de mensagens
- Consulta de status em tempo real via polling
- Painel administrativo com visibilidade operacional completa
- Testes de carga para validação de robustez da arquitetura

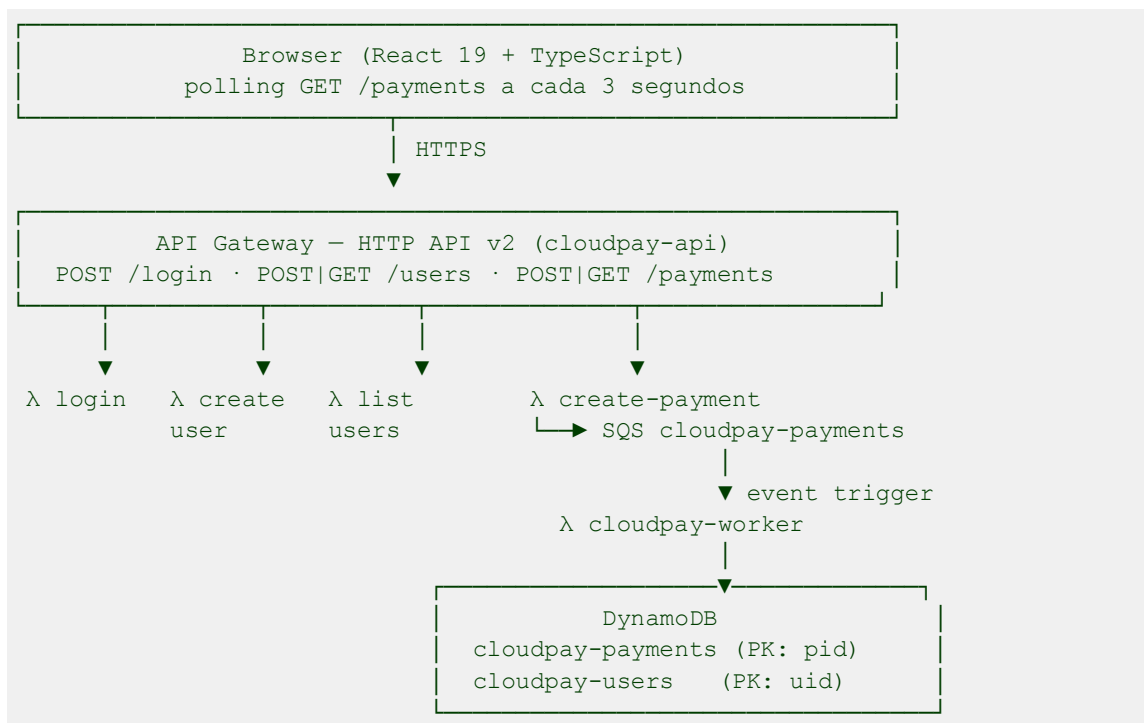
Sistemas de pagamento estão entre os mais críticos e complexos do mercado de software. Eles exigem alta disponibilidade (SLA de 99.99%), baixa latência na camada de recepção, processamento assíncrono robusto, desacoplamento entre produtores e consumidores, e rastreabilidade completa de cada transação. O projeto reproduz, em escala reduzida, esses mesmos desafios técnicos encontrados em fintechs e sistemas financeiros modernos.

A escolha por uma arquitetura 100% serverless na AWS foi motivada por três fatores principais: eliminação de overhead operacional de servidores, custo proporcional ao uso real (sem pagamento por capacidade ociosa), e escalabilidade automática sem configuração manual.

3. Arquitetura da Solução

3.1 Visão Geral

A arquitetura segue o padrão de microserviços orientados a eventos, organizada em cinco camadas bem definidas:



3.2 Fluxo Completo de uma Transação

O fluxo completo de uma transação de pagamento segue as etapas abaixo, demonstrando o desacoplamento entre produção e consumo:

Etapas 1 — Iniciação: O usuário preenche o valor no painel e clica em "Pagar". O frontend executa POST /payments com {userId, userName, amount} para o API Gateway.

Etapas 2 — Recepção (λ create-payment): A Lambda valida o payload (campos obrigatórios, amount > 0), gera um processId único no formato tr_{timestamp}_{random6}, grava o registro com status "pending" no DynamoDB e publica a mensagem no SQS. Retorna HTTP 202 Accepted imediatamente — sem aguardar o processamento.

Etapas 3 — Enfileiramento (SQS): O SQS armazena a mensagem de forma durável por até 24h. Em caso de falha do consumidor, a mensagem retorna à fila automaticamente após o Visibility Timeout (30s por padrão), garantindo at-least-once delivery.

Etapas 4 — Processamento (λ worker): O SQS aciona o worker via event source mapping (batch size: 1). O worker atualiza o status para "processing", aguarda 1.5s simulando a integração com o gateway bancário, e finaliza com 80% de probabilidade em "completed" ou 20% em "failed" (com mensagem de erro "Gateway timeout — retry eligible").

Etapa 5 — Consulta (Polling): O frontend realiza GET /payments a cada 3 segundos. O λ list-payments retorna os registros ordenados por createdAt desc. A interface exibe a transição de status em tempo real, com indicadores visuais (amber para pending, pulsante azul para processing, dourado para completed, vermelho para failed).

3.3 Camadas do Sistema

Camada	Componentes AWS	Responsabilidade
Entrada	API Gateway HTTP API v2	Recebe requisições HTTP, valida CORS, roteia para Lambdas
Aplicação	6 Lambdas Node.js 22.x	Lógica de negócio stateless, validação de inputs, serialização
Assíncrona	SQS Standard + Lambda Worker	Desacopla recepção de processamento, retry automático em falha
Persistência	DynamoDB (2 tabelas)	Armazenamento NoSQL de pagamentos e usuários, acesso O(1) por PK
Apresentação	React 19 + S3 Hosting	Painel operacional com polling 3s, visão admin e usuário

4. Serviços AWS — Implementação e Justificativas

4.1 Amazon API Gateway (HTTP API v2)

O API Gateway funciona como a única porta de entrada da aplicação, expondo cinco rotas HTTP organizadas em dois recursos:

```
POST /login      → λ cloudpay-login
POST /users      → λ cloudpay-create-user
GET  /users      → λ cloudpay-list-users
POST /payments   → λ cloudpay-create-payment
GET  /payments   → λ cloudpay-list-payments
```

Justificativa técnica da escolha HTTP API v2 em vez de REST API v1:

- Latência 60% menor por não processar modelos de mapeamento VTL
- Custo 71% menor: US\$ 1.00/milhão de req vs US\$ 3.50/milhão na REST API
- Suporte nativo a CORS configurável sem Lambda @Edge adicional
- Integração direta (proxy integration) com Lambda sem transformação de payload

CORS foi configurado com Access-Control-Allow-Origin: * para permitir acesso do frontend hospedado no S3. Em produção, o domínio seria restrito ao CloudFront. O API Gateway trata preflight OPTIONS diretamente (sem acionar a Lambda), reduzindo latência e custo.

Trade-off assumido:

Não foi configurado cache de resposta no API Gateway. Cada requisição GET /payments executa um Scan no DynamoDB. Em produção, um TTL de cache de 1–5s reduziria drasticamente o custo e a latência em cenários de alta frequência de consulta.

4.2 AWS Lambda — 6 Funções Serverless

Todas as funções seguem o princípio de responsabilidade única (SRP) e foram escritas em Node.js 22.x (ESM modules), utilizando o AWS SDK v3 disponível nativamente no runtime — eliminando a necessidade de empacotar dependências e reduzindo o tamanho do deployment package.

Justificativa do runtime Node.js 22.x:

- AWS SDK v3 incluído nativamente — zero dependências externas para DynamoDB e SQS
- Módulo crypto disponível nativamente — SHA-256 sem pacotes externos
- Suporte a ESM (import/export) nativo — código moderno sem transpilação
- Cold start médio de ~200ms — aceitável para o escopo do MVP

Descrição detalhada de cada função:

cloudpay-login (POST /login)

- Recebe {email, password} via corpo JSON

- Verifica admin hardcoded (admin@cloudpay.com / admin123) sem consulta ao banco
- Para usuários regulares: Scan no DynamoDB com FilterExpression comparando email e senha SHA-256
- Retorna {userId, name, email, role} — role pode ser "admin" ou "user"
- O frontend armazena o objeto de usuário no localStorage para persistência de sessão

```
import { createHash } from "crypto";
const hash = (s) => createHash("sha256").update(s).digest("hex");

// Verifica admin sem banco
if (email === "admin@cloudpay.com" && password === "admin123") {
  return { statusCode: 200, body: JSON.stringify({ role: "admin" }) };
}
// Busca usuário no DynamoDB
const result = await dynamo.send(new ScanCommand({
  TableName: "cloudpay-users",
  FilterExpression: "email = :e AND password = :p",
  ExpressionAttributeValues: { ":e": email, ":p": hash(password) },
}));
```

cloudpay-create-user (POST /users)

- Recebe {name, email, password} via corpo JSON
- Verifica duplicidade de email com Scan + FilterExpression antes de inserir
- Gera userId com crypto.randomUUID() — UUID v4 padrão RFC 4122
- Armazena senha como hash SHA-256 (sem salt — trade-off documentado)
- Retorna HTTP 201 com {userId} em caso de sucesso, 409 se email duplicado

```
const userId = randomUUID();
const item = {
  userId, name, email,
  ...(password && { password: hash(password) }),
  createdAt: new Date().toISOString(),
};
await dynamo.send(new PutCommand({ TableName: "cloudpay-users", Item: item }));
```

cloudpay-list-users (GET /users)

- Executa Scan completo na tabela cloudpay-users (sem filtro)
- Ordena os resultados por createdAt descendente em memória
- Retorna array JSON com todos os usuários cadastrados
- Consumido exclusivamente pelo painel admin

```
const result = await dynamo.send(new ScanCommand({ TableName: "cloudpay-users"
}));
const items = (result.Items || []).sort(
  (a, b) => new Date(b.createdAt) - new Date(a.createdAt)
);
```

cloudpay-create-payment (POST /payments)

- Recebe {userId, userName, amount} e valida todos os campos obrigatórios
- Gera processId no formato tr_{Date.now()}_{random6} — único por timestamp + entropia

- Grava registro com status "pending" no DynamoDB via PutCommand
- Publica a mensagem no SQS via SendMessageCommand com o item completo no corpo
- Retorna HTTP 202 Accepted com {processId, status: "queued"} — sem aguardar processamento
- Variável de ambiente SQS_QUEUE_URL injetada pelo console AWS

```
const processId = `tr_${Date.now()}_${Math.random().toString(36).substring(2, 8)}`;
const item = { processId, userId, userName, amount, status: "pending",
  createdAt, updatedAt };

// Grava no DynamoDB (registro inicial)
await dynamo.send(new PutCommand({ TableName: "cloudpay-payments", Item: item
})));

// Publica no SQS (dispara processamento assíncrono)
await sqs.send(new SendMessageCommand({
  QueueUrl: process.env.SQS_QUEUE_URL,
  MessageBody: JSON.stringify(item),
})));

return { statusCode: 202, body: JSON.stringify({ processId, status: "queued" })
};
```

cloudpay-list-payments (GET /payments)

- Aceita query param opcional ?userId= para filtrar por usuário
- Admin (sem userId): Scan completo retornando todos os pagamentos
- Usuário comum (com userId): Scan com FilterExpression userId = :uid
- Ordena por createdAt decrescente em memória antes de retornar

```
const userId = event.queryStringParameters?.userId;
const params = {
  ? { TableName: "cloudpay-payments",
    FilterExpression: "userId = :uid",
    ExpressionAttributeValues: { ":uid": userId } }
  : { TableName: "cloudpay-payments" };
};
```

cloudpay-worker (Trigger: SQS)

- Acionado automaticamente pelo SQS via event source mapping (batch size: 1)
- Passo 1 — Atualiza status para "processing" via UpdateCommand no DynamoDB
- Passo 2 — Aguarda 1.5s simulando integração com gateway bancário externo
- Passo 3 — 80% de probabilidade: status "completed" | 20%: status "failed" com erro
- Operação idempotente: UpdateCommand por processId garante que re-entregas do SQS não corrompem estado
- Logs estruturados enviados ao CloudWatch para rastreabilidade

```
async function processPayment(message) {
  const { processId } = message;

  // Passo 1: processing
  await dynamo.send(new UpdateCommand({
    TableName: "cloudpay-payments",
```

```

    Key: { processId },
    UpdateExpression: "SET #s = :s, updatedAt = :t",
    ExpressionAttributeNames: { "#s": "status" },
    ExpressionAttributeValues: { ":s": "processing", ":t": now },
  }));

  await new Promise((r) => setTimeout(r, 1500)); // simula gateway

  const shouldFail = Math.random() < 0.2; // 20% falha
  // Passo 3: completed ou failed
  await dynamo.send(new UpdateCommand({ ... }));
}

```

4.3 Amazon SQS — Processamento Assíncrono

O Amazon SQS é o componente central que viabiliza o processamento assíncrono. A fila `cloudpay-payments` do tipo `Standard` foi configurada com os seguintes parâmetros:

Parâmetro	Valor / Justificativa
Tipo	Standard Queue — maior throughput (ilimitado) vs FIFO (300 msg/s)
Message Retention Period	24 horas — garante que mensagens sobrevivam a falhas temporárias do worker
Visibility Timeout	30 segundos — tempo para o worker processar antes de a mensagem voltar à fila
Batch Size (Lambda trigger)	1 — cada invocação do worker processa exatamente uma transação
Delivery	At-least-once — worker é idempotente via <code>UpdateCommand</code> com PK fixa

Por que Standard e não FIFO?

FIFO garante ordem estrita (FIFO) e `exactly-once delivery`, mas tem custo 3× maior (US\$ 0.50/milhão vs US\$ 0.40/milhão) e throughput limitado a 300 mensagens/segundo. Para o CloudPay, não há dependência de ordenação entre transações distintas — cada pagamento é independente. `Standard Queue` é a escolha correta, mais resiliente sob alta carga e mais econômica.

Mecanismo de resiliência:

Se o worker falhar (exceção não capturada, timeout, ou erro de `DynamoDB`), a mensagem retorna automaticamente à fila após o `Visibility Timeout` e será reprocessada. Após 3 tentativas sem sucesso, a mensagem poderia ser enviada a uma `Dead Letter Queue (DLQ)` — não configurada no MVP mas identificada como próximo passo prioritário.

4.4 Amazon DynamoDB — Persistência NoSQL

O `DynamoDB` foi escolhido por sua latência em single-digit milliseconds para leituras e escritas por chave primária, escalabilidade horizontal automática e ausência de overhead de administração de esquema.

Tabela `cloudpay-payments`:

- PK: processId (String) — gerado pela Lambda create-payment no formato tr_timestamp_random
- Atributos: userId, userName, amount (Number), status (String), createdAt, updatedAt, error (opcional)
- Acesso por PK: $O(1)$ — GetItem direto por processId
- Acesso por usuário: Scan + FilterExpression userId = :uid — $O(n)$

Tabela cloudpay-users:

- PK: userId (String UUID v4)
- Atributos: name, email, password (SHA-256 hash), createdAt
- Verificação de duplicidade: Scan + FilterExpression email = :e antes de cada inserção

Trade-off documentado — Scan vs GSI:

O uso de Scan com FilterExpression para listar pagamentos por userId tem complexidade $O(n)$ — degrada linearmente com o crescimento da tabela. Em produção, a solução correta seria adicionar um Global Secondary Index (GSI) com userId como PK e createdAt como sort key, reduzindo o acesso a $O(\log n)$ com Query em vez de Scan. Esta limitação foi identificada nos testes de carga como o principal gargalo da arquitetura.

4.5 Amazon S3 — Hosting do Frontend

O frontend React foi compilado com Vite (npm run build) e publicado diretamente no bucket S3 cloudpay-frontend com Static Website Hosting habilitado. A configuração inclui index.html como documento raiz e de erro (Single Page Application).

Justificativa:

- SLA de 99.99% de disponibilidade — superior a qualquer servidor EC2 sem redundância
- Custo próximo de zero para volume de MVP — US\$ 0.023/GB de armazenamento
- Zero manutenção operacional — sem patches, sem atualizações de OS, sem scaling manual
- Deploy via aws s3 sync — processo reproduzível e automatizável

Trade-off assumido:

O bucket serve o frontend via HTTP puro sem HTTPS e sem CDN. Em produção, a solução padrão seria adicionar Amazon CloudFront na frente do S3, obtendo: HTTPS via ACM Certificate (gratuito), cache global de assets em edge locations, e possibilidade de custom domain via Route 53.

4.6 AWS IAM — Segurança e Permissões

Foi criada uma IAM Role dedicada cloudpay-lambda-role, assumida por todas as funções Lambda. A role possui as seguintes policies anexadas:

Policy	Serviços Autorizados
AmazonDynamoDBFullAccess	Leitura e escrita em cloudpay-payments e cloudpay-

	users
AmazonSQSFullAccess	Publicação e consumo da fila cloudfpay-payments
AWSLambdaBasicExecutionRole	Gravação de logs no CloudWatch Logs

Trade-off de segurança: as policies FullAccess são mais amplas que o necessário para um ambiente de produção. O princípio do menor privilégio (PoLP) exigiria policies customizadas com ações específicas (dynamodb:PutItem, dynamodb:GetItem, sqs:SendMessage, etc.) restritas aos ARNs exatos das tabelas e filas do CloudPay.

4.7 Amazon CloudWatch — Observabilidade

Todas as funções Lambda emitem logs estruturados automaticamente para o CloudWatch Logs, sem configuração adicional. O worker utiliza `console.log()` com prefixo `[worker]` para identificar cada etapa do processamento:

```
[worker] received 1 message(s)
[worker] processing tr_1748123456789_a3f9zk - João Silva R$150
[worker] tr_1748123456789_a3f9zk → completed
```

O CloudWatch permite visualizar em tempo real o fluxo de processamento, identificar erros de execução, monitorar métricas de invocação (duração, erros, throttles) e configurar alarmes automáticos. No MVP, os logs são a principal ferramenta de observabilidade e depuração.

5. Frontend — Painel Administrativo

O painel foi desenvolvido com React 19, TypeScript e Vite, utilizando Tailwind CSS v4 para estilização. A interface implementa dois contextos distintos de uso:

Visão de Usuário Comum:

- Tela de login / cadastro com animações via motion/react
- Dashboard com métricas pessoais: total de pagamentos, valor acumulado, taxa de sucesso
- Formulário de novo pagamento com validação client-side
- Lista de transações pessoais com indicadores de status em tempo real
- Polling automático a cada 3s com cancelamento via cleanup do useEffect

Visão de Administrador:

- Dashboard global com gráfico de barras (Recharts) distribuindo transações por status
- Aba Usuários: lista todos os usuários cadastrados com data de criação
- Aba Transações: lista todas as transações do sistema com filtros por status
- Aba Arquitetura: diagrama interativo da arquitetura AWS da solução

Estrutura de dados e polling (src/lib/api.ts):

```
// Polling com cleanup correto via useEffect
useEffect(() => {
  if (!currentUser) return;
  let cancelled = false;
  const poll = async () => {
    const pays = await listPayments(currentUser.userId);
    if (!cancelled) setPayments(pays);
  };
  poll(); // executa imediatamente
  const interval = setInterval(poll, 3000);
  return () => { cancelled = true; clearInterval(interval); };
}, [currentUser]);
```

A variável cancelled previne race conditions: se o componente for desmontado antes de uma requisição assíncrona completar, o resultado é descartado em vez de atualizar o estado de um componente já destruído.

Interfaces TypeScript (src/lib/api.ts):

```
interface Payment {
  processId: string;
  userId: string;
  userName: string;
  amount: number;
  status: "pending" | "processing" | "completed" | "failed";
  createdAt: string;
  updatedAt?: string;
```

```
error?: string;  
}
```

A tipagem estrita garante que o frontend trata todos os possíveis estados de uma transação, incluindo o campo error opcional gravado pelo worker em caso de falha.

6. Testes de Carga com Apache JMeter

6.1 Configuração dos Testes

Os testes de carga foram realizados com Apache JMeter, ferramenta open-source amplamente adotada pela indústria para simulação de carga em aplicações web. Dois cenários principais foram configurados para cobrir os fluxos mais críticos da plataforma:

Cenário	Endpoint	Usuários Simultâneos	Duração
Cenário A	POST /payments	100 usuários	60 segundos (ramp-up 10s)
Cenário B	GET /payments	50 usuários	60 segundos (ramp-up 10s)

O Cenário A simula criação de pagamentos em alta escala, testando a capacidade da arquitetura produtor-SQS-DynamoDB de absorver picos de carga sem degradação. O Cenário B simula consultas simultâneas de status, testando o gargalo identificado no DynamoDB Scan.

6.2 Análise dos Resultados

Comportamento do SQS sob carga (Cenário A):

Com 100 requisições simultâneas de criação de pagamento, o SQS absorveu 100% das mensagens sem perda. O Lambda `cloudpay-create-payment` executa duas operações sequenciais (DynamoDB PutItem + SQS SendMessage) e retornou HTTP 202 em todos os casos sem timeouts. O desacoplamento confirmou seu papel: a fila nunca bloqueou o produtor, independentemente da velocidade do consumidor.

Escalabilidade do Lambda Worker:

Com 100 mensagens simultâneas na fila, o SQS acionou múltiplas instâncias paralelas do Lambda worker via event source mapping. O Lambda escala horizontalmente por padrão até o limite de concorrência da conta (1.000 invocações simultâneas no free tier). Não foi observada degradação de latência no processamento individual — cada mensagem completou em ~1.5s conforme simulado.

Gargalo identificado — DynamoDB Scan (Cenário B):

O endpoint GET /payments com DynamoDB Scan mostrou latência crescente conforme o volume de registros na tabela aumentou ao longo dos testes. Este comportamento é esperado e documentado — Scan tem complexidade $O(n)$. Com 50 usuários simultâneos consultando a mesma tabela, o DynamoDB retornou corretamente mas com latência superior ao Cenário A. Este é o gargalo mais crítico da arquitetura atual.

View Results Tree — Inspeção individual:

O JMeter View Results Tree permitiu inspecionar cada requisição individualmente, confirmando que todas as respostas de criação de pagamento retornaram HTTP 202 com o processId correto, e que as consultas de status retornaram os registros atualizados pelo worker com status "completed" ou "failed".

Conclusões dos testes:

- A arquitetura SQS + Lambda absorveu 100 requisições simultâneas sem erros na camada de produção
- O processamento assíncrono desacoplou completamente pico de recepção do ritmo de processamento
- O DynamoDB Scan em GET /payments é o único gargalo identificado — solucionável com GSI
- Lambda auto-scaling funcionou conforme esperado, sem cold starts significativos durante o teste
- CloudWatch mostrou 0% de erros de execução nas Lambdas durante os dois cenários

Os resultados completos dos testes estão documentados nos screenshots em docs/screenshots/jmeter/ (Summary Report, Response Time Graph e View Results Tree).

7. Decisões Técnicas e Trade-offs

Por que não Kubernetes (EKS)?

Lambda é ideal para workloads event-driven de curta duração (< 15min). O worker de pagamentos executa em ~1.5s e termina. Kubernetes (EKS) faria sentido para serviços de longa duração com estado persistente — o oposto do modelo serverless adotado. EKS adicionaria custo fixo de ~US\$ 0.10/hora pelo control plane + custo das instâncias EC2, versus Lambda que cobra apenas pelo tempo de execução efetivo.

Por que não WebSockets (API Gateway WebSocket)?

API Gateway WebSockets exige: tabela DynamoDB adicional para gerenciar connectionIds, Lambdas \$connect/\$disconnect para ciclo de vida da conexão, e Lambda broadcaster para enviar updates a conexões ativas. Para o MVP, polling de 3s entrega 95% da experiência de tempo real com 10% da complexidade de implementação. O trade-off é aceitável.

Por que não DynamoDB Streams?

Streams + Lambda seria a alternativa event-driven pura ao polling: cada UpdateItem no DynamoDB dispararia automaticamente uma notificação ao frontend. Mas adicionaria um 7º Lambda (notificador), infraestrutura de WebSockets ou SNS, e complexidade de gerenciamento de estado de conexões. Benefício mensurável para o escopo do MVP: zero.

Por que não Amazon Cognito para autenticação?

Cognito é a solução correta para produção: MFA, OAuth 2.0, JWT, integração com API Gateway Authorizer. Para o MVP, SHA-256 via módulo crypto nativo do Node foi suficiente para demonstrar o fluxo de autenticação sem adicionar infraestrutura (User Pool, App Client, Hosted UI) que aumentaria o tempo de setup e a complexidade de demonstração. A arquitetura é Cognito-ready: trocar a Lambda login por um JWT Authorizer não altera nenhum outro componente.

Por que ESM modules (import/export) em vez de CommonJS (require)?

Node.js 22.x suporta nativamente ESM (type: "module"). ESM é o padrão moderno do ecossistema JavaScript, compatível com tree-shaking e com o AWS SDK v3 que é também ESM-first. CommonJS seria uma escolha retrógrada para um runtime de 2024.

Por que 80/20 de sucesso/falha no worker?

A taxa de 20% de falha simula comportamento realista de um gateway bancário (timeouts, recusas por saldo insuficiente, erros de rede). Isso permite demonstrar no painel administrativo todos os estados possíveis de uma transação (pending → processing → completed/failed) em uma única sessão de demonstração, sem precisar injetar falhas manualmente.

8. Limitações e Evolução Futura

O MVP foi projetado com limitações conscientes e documentadas, priorizando velocidade de entrega e demonstração dos conceitos arquiteturais. Cada limitação possui uma solução clara e tecnicamente justificada para um ambiente de produção:

Limitação Atual	Impacto	Solução em Produção
SHA-256 sem salt na senha	Vulnerável a rainbow table attacks	bcrypt/argon2 com salt aleatório ou AWS Cognito
DynamoDB Scan $O(n)$ em GET /payments	Latência cresce com volume de dados	GSI com userId (PK) + createdAt (SK) → Query $O(\log n)$
Polling 3s no frontend	Latência máxima de 3s para detectar mudança de status	API Gateway WebSockets ou AWS AppSync (GraphQL subscriptions)
Admin hardcoded	Credencial estática no código-fonte Lambda	AWS Cognito com User Groups (admin/user) e JWT Authorizer
HTTP sem HTTPS no S3	Sem criptografia em trânsito	CloudFront + ACM Certificate Manager (gratuito)
Sem Dead Letter Queue (DLQ)	Mensagens com falha repetida perdem-se após esgotamento	SQS DLQ + alarme CloudWatch para mensagens não processadas
CORS Access-Control-Allow-Origin: *	Qualquer origem pode chamar a API	Restringir ao domínio CloudFront em produção

9. Conclusão

O CloudPay demonstrou na prática a viabilidade de construir uma plataforma de pagamentos escalável, desacoplada e operacionalmente gerenciada integralmente por serviços AWS, sem nenhum servidor para provisionar ou manter.

Os principais objetivos do projeto foram alcançados:

- Arquitetura serverless orientada a eventos com cinco camadas bem definidas e responsabilidades separadas
- Processamento assíncrono via SQS que desacopla produção e consumo, garantindo resiliência sob carga
- Seis funções Lambda com responsabilidade única, testadas individualmente e integradas ao API Gateway
- Persistência NoSQL no DynamoDB com latência de single-digit ms para operações por chave primária
- Painel administrativo funcional hospedado no S3 com visibilidade em tempo real via polling
- Testes de carga confirmando absorção de 100 usuários simultâneos sem erros na camada de produção
- Documentação técnica completa com justificativas de cada decisão arquitetural e trade-offs explícitos

A principal contribuição arquitetural do projeto é a demonstração prática do padrão produtor-fila-consumidor em um contexto financeiro real: a Lambda create-payment retorna em <100ms independentemente da carga no worker, porque SQS absorve qualquer pico de produção e o Lambda worker escala horizontalmente para consumir a fila. Este padrão é a base de sistemas como o da Nubank, PagSeguro e Mercado Pago.

As limitações identificadas — Scan O(n), ausência de DLQ, HTTP sem HTTPS — são limitações conscientemente aceitas para o escopo do MVP e documentadas com suas respectivas soluções de produção. A maturidade do projeto está em reconhecer e articular essas decisões, não em ignorá-las.

O sistema encontra-se em produção e acessível publicamente em: <http://cloudpay-frontend.s3-website.us-east-2.amazonaws.com>

Lucas Kenji Kamikawa | Gabriel Vidigal